Architectural Plan & Technical Design: IBM i RPG Source Extraction Utility (v2)

1. Executive Summary & Objective

The objective is to deliver an enterprise-grade, modular IBM i utility (`EXTRPGSRC`) that accepts between 1 and 10 library names, validates pre-execution that all specified libraries exist, scans each library independently for all eligible RPG-related source members (regardless of source file name), extracts the source lines with full metadata, and consolidates the records into a single Physical File (`EXTRPGPF`).

2. Technical Evaluation & Architecture

2.1 Command Parameter Design & Min/Max Enforcement

Feature	Design A: List Parameter (`KWD(LIB) MAX(10)`)	Design B: Individual Keywords (`LIB1` ... `LIB10`)
Command Syntax	`EXTRPGSRC LIB(ERP HR SVC)`	`EXTRPGSRC LIB1(ERP) LIB2(HR) LIB3(SVC)`
Min/Max Validation	Enforced natively by Command Analyzer (`MIN(1) MAX(10)`)	Enforced by declaring `LIB1` as `MIN(1)` , `LIB2..10` as `DFT(*NONE)`
User Experience (F4)	Single list entry box with expandable lines (`+` for more)	10 separate parameter prompt fields
Memory Buffer	Passes 2-byte count integer followed by 10-char elements	Passes 10 separate `*CHAR 10` parameters
Standard Precedent	Standard IBM i convention (like `SAVLIB` , `DSPOBJD`)	Legacy utility style

Recommendation & Decision: We implement **Design A** (`PARM KWD(LIB) TYPE(*NAME) LEN(10) MIN(1) MAX(10)`) because OS/400 Command Analyzer natively blocks 0 libraries or >10 libraries before invocation.

Memory Buffer Layout in CLLE

When `TYPE(*NAME) LEN(10) MIN(1) MAX(10)` is passed to both `CHKLIBS` and `EXTRPGCL`:

- Bytes 1–2: `*INT2` (halfword integer) representing the number of entered libraries (1 to 10).
- Bytes 3–12: 1st library name (left-aligned, blank padded to 10 chars).
- Bytes 13–22: 2nd library name.
- Bytes 23–32: 3rd library name, and so on up to 102 bytes total (`2 + (10 * 10)`).

2.2 Pre-Execution Library Verification via Validity Checking Program (`VLDPGM: CHKLIBS`)

The user specifically requested validating all entered libraries **before** calling the main command processing program, stopping execution immediately with an explicit message if any library does not exist.

How IBM i Validity Checking Programs Work

1. When a user runs `EXTRPGSRC LIB(...)`, the Command Analyzer parses parameter syntax and enforces `MIN(1)` and `MAX(10)`.
2. Before calling the Command Processing Program (`PGM(EXTRPGCL)`), the Command Analyzer checks if `VLDPGM` is defined on `CRTCMD`.
3. If `VLDPGM(CHKLIBS)` is specified, the Command Analyzer invokes `CHKLIBS`, passing the **exact same parameter list** that would be passed to `EXTRPGCL`.
4. In `CHKLIBS`:
    - It unpacks the count and iterates through each library name.
    - For each library, it executes:

      ```
      cl
      CHKOBJ OBJ(&CURLIB) OBJTYPE(*LIB)
      ```

    - **If the library does not exist (`CPF9801`)**: It issues a diagnostic message (`*DIAG`):

      ```
      cl
      SNDPGMMSG MSGID(CPD0006) MSGF(QCPFMSG) +
                MSGDTA('0000Library ' *CAT &CURLIB *TCAT ' does not exist.') +
                MSGTYPE(*DIAG)
      ```

      and sets an error flag `&HASERROR = '1'`.

- **If the user is not authorized to the library (`CPF9802`)**: It issues a diagnostic message (`*DIAG`):

  ```
  cl
  SNDPGMMSG MSGID(CPD0006) MSGF(QCPFMSG) +
            MSGDTA('0000Not authorized to library ' *CAT &CURLIB *TCAT '.') +
            MSGTYPE(*DIAG)
  ```

  and sets `&HASERROR = '1'`.
- It also validates that the output library specified in `OUTLIB` exists.

5. **Termination Behavior**:
   - If any library failed validation (`&HASERROR = '1'`), `CHKLIBS` sends the standard IBM i escape message:

     ```
     cl
     SNDPGMMSG MSGID(CPF0002) MSGF(QCPFMSG) MSGTYPE(*ESCAPE)
     ```

     `CPF0002` ("Error found by &1 during execution of &2") signals the Command Analyzer that validity checking failed.
   - **The Command Analyzer halts immediately: `EXTRPGCL` is NEVER called.**
   - In interactive mode (command line or F4 prompt), the prompt displays the exact diagnostic messages at the bottom of the display and highlights the parameter. In batch mode, the job step fails cleanly.
   - If all libraries are valid, `CHKLIBS` exits normally, and the Command Analyzer proceeds to call `EXTRPGCL`.

2.3 Source Member Discovery Mechanism

In IBM i, source members can live in any physical file where `FILETYPE = *SRC`, not just `QRPGSRC` or `QRPGLESRC` (e.g., `APPSRCS`, `DEVSRC`, `PROD_SRC`, `MYPGMS`).

Discovery Strategy: `DSPFD TYPE(*MBRLIST)` to `QTEMP`

For each validated library:

1. `EXTRPGCL` executes:

```
cl
DSPFD FILE(&LIB/*ALL) TYPE(*MBRLIST) OUTPUT(*OUTFILE) +
      OUTFILE(QTEMP/QAFDMBL) OUTMBR(*FIRST *REPLACE)
```

2. `DSPFD` inspects the entire library and generates the IBM standard model file `QAFDMBL` (format `QWHFDMBL`) in `QTEMP`.
3. Key fields captured:
    - `MLFILE`: Physical file name
    - `MLLIB`: Library name
    - `MLNAME`: Source member name
    - `MLFTYP`: File type (`'S'` for Source physical file, `'D'` for Data file)
    - `MLSEU2`: Source type (e.g., `RPGLE`, `SQLRPGLE`, `RPG`, `CLLE`, `DDS`)
4. **Empty Library Handling**:
    - If a library contains no database files or no members, `DSPFD` sends `CPF3012` or `CPF3033`.
    - `EXTRPGCL` captures these with `MONMSG`, logs an informational message in the job log ("Library &LIB contains no source members"), and continues to the next library without failing.

2.4 Member Eligibility Criteria & Maintainability

The utility must identify RPG-related source members based on member source type (`MLSEU2`), NOT the containing file name.

Eligible Source Types Matrix:

- `RPGLE`: ILE RPG (both fixed-form and fully free-form `**FREE`)
- `SQLRPGLE`: ILE RPG with embedded SQL
- `RPG`: Classic RPG III / RPG 400
- `SQLRPG`: Classic RPG III with embedded SQL
- `RPG38`: System/38 RPG III source
- `RPGP`: RPG Program source

Encapsulated Eligibility Procedure in RPGLE:

```
rpgle
dcl-proc IsEligibleType;
  dcl-pi *n ind;
    pType char(10) const;
  end-pi;

  dcl-s cleanType varchar(10);
  dcl-s i int(10);
  dcl-c ELIGIBLE_COUNT const(6);
  dcl-s typeList char(10) dim(ELIGIBLE_COUNT) ctdata;

  cleanType = %trim(pType);

  for i = 1 to ELIGIBLE_COUNT;
    if cleanType = %trim(typeList(i));
      return *on;
    endif;
  endfor;

  return *off;
end-proc;
**CTDATA typeList
RPGLE
SQLRPGLE
RPG
SQLRPG
RPG38
RPGP
```

Adding new types (e.g., `RPGMOD`) requires simply updating the compile-time array or adding an entry to this routine.

2.5 Source Record Extraction (Overcoming Level Checks & Variable Lengths)

Traditional RPGLE `F`-spec reading requires an externally described file or program-described file with fixed buffer lengths. If source physical files across libraries have different record format names or record lengths (e.g., 92, 112, 132, 240 bytes), traditional `OVRDBF` causes **Format Level Check (`CPF4131`)** or data truncation.

Solution: DB2 for i Dynamic SQL Member Aliasing

For each eligible member discovered:

1. RPGLE generates a dynamic SQL alias in `QTEMP`:

```sql
CREATE OR REPLACE ALIAS QTEMP.CURSRCMBR FOR "LIBRARY"."FILE" ("MEMBER");
```

2. DB2 for i surfaces the member as a standard relational table with columns `SRCSEQ`, `SRCDAT`, and `SRCDTA`.
3. RPGLE executes a single set-based SQL insert:

```sql
INSERT INTO OUTLIB/EXTRPGPF
  (SRCLIB, SRCFILE, SRCMBR, SRCTYPE, SRCSEQ, SRCDAT, SRCDTA)
SELECT
  :curLib, :curFile, :curMbr, :curType, SRCSEQ, SRCDAT, SRCDTA
FROM QTEMP.CURSRCMBR;
```

Key Advantages:

- **Zero Level Checks**: `CREATE ALIAS` bypasses record format level checks entirely.
- **Automatic Padding & Truncation**: DB2 promotes source data of any width (80, 100, 112, 132, 240) into the consolidated output column cleanly.
- **Database Engine Speed**: Block insertion directly inside DB2 without row-by-row RPG buffer moves.
- **Empty Member Handling**: If a member has 0 lines, `INSERT ... SELECT` inserts 0 rows and returns `SQLCODE = 100` cleanly without error.

2.6 Consolidated Output Physical File (PF) Architecture

The consolidated physical file `EXTRPGPF` contains:

Field Name	Type	Length / Dec	Description
`SRCLIB`	CHAR	10	Source library where member originated
`SRCFILE`	CHAR	10	Source physical file name (e.g. `QRPGLESRC`, `APPSRCS`)
`SRCMBR`	CHAR	10	Source member name
`SRCTYPE`	CHAR	10	Source type (e.g. `RPGLE`, `SQLRPGLE`, `RPG`)
`SRCSEQ`	ZONED	6, 2	Source sequence number
`SRCDAT`	ZONED	6, 0	Source modification date (YYMMDD)
`SRCDTA`	CHAR	256	Source text statement (accommodates up to 240-char free-form)
`EXTRTIME`	TIMESTAMP	-	Extraction audit timestamp

Initialization Behavior (`CLEAR` parameter):

- Parameter `CLEAR(*YES | *NO)` defaults to `*YES`.
- When `*YES`: `EXTRPGCL` runs `CLRPFM FILE(&OUTLIB/&OUTFILE)` before starting the extraction loop.
- When `*NO`: New extraction records are appended to existing data.

3. Component Architecture & Responsibility Matrix

Component	Source Member	Language / Type	Primary Responsibilities
Command	`EXTRPGSRC`	`*CMD`	Defines parameters (`LIB` , `OUTLIB` , `OUTFILE` , `CLEAR`). Enforces 1–10 libraries. Binds `VLDPGM(CHKLIBS)` and `PGM(EXTRPGCL)` .
Validity Program	`CHKLIBS`	`*PGM` (CLLE)	Pre-execution validator. Validates that ALL entered libraries exist and are accessible. Sends `*DIAG` + `CPF0002` if any library is invalid. Blocks CPP invocation.
Controller Program	`EXTRPGCL`	`*PGM` (CLLE)	Coordinates execution. Clears output PF (if `CLEAR(*YES)`). Loops through libraries. Runs `DSPFD *MBRLIST` to `QTEMP` . Calls RPGLE extraction engine. Emits final summary message.
Extraction Engine	`EXTRPGRG`	`*PGM` (SQLRPGLE)	Filters `QTEMP/QAFDMBL` for eligible RPG types (`MLFTYP='S'` , `MLSEU2`). Creates dynamic aliases for each member. Performs set-based `INSERT ... SELECT` . Returns metrics.
Consolidated PF	`EXTRPGPF`	`*FILE` (DDS / SQL)	Physical file storing all consolidated source records and metadata.

4. Sequence of Execution

5. Compilation Order & Dependencies

Due to object references on IBM i, components must be created in this strict order:

```
Step 1: EXTRPGPF (DDS Physical File)
   │
Step 2: EXTRPGRG (SQLRPGLE Program - references EXTRPGPF)
   │
Step 3: EXTRPGCL (CLLE Program - references EXTRPGPF and EXTRPGRG)
   │
Step 4: CHKLIBS  (CLLE Program - validity checking program)
   │
Step 5: EXTRPGSRC (CMD - binds PGM(EXTRPGCL) and VLDPGM(CHKLIBS))
```

Detailed Compilation Commands:

```
cl
/* 1. Create Output Physical File */
CRTPF FILE(&CURLIB/EXTRPGPF) SRCFILE(&CURLIB/QDDSSRC) SRCMBR(EXTRPGPF) +
      TEXT('Consolidated RPG Extracted Source Records')

/* 2. Compile RPGLE Extraction Engine */
CRTSQLRPGI OBJ(&CURLIB/EXTRPGRG) SRCFILE(&CURLIB/QRPGLESRC) SRCMBR(EXTRPGRG) +
           COMMIT(*NONE) OBJTYPE(*PGM) DBGVIEW(*SOURCE)

/* 3. Compile CLLE Controller Program */
CRTBNDCL PGM(&CURLIB/EXTRPGCL) SRCFILE(&CURLIB/QCLLESRC) SRCMBR(EXTRPGCL) +
         DBGVIEW(*SOURCE)

/* 4. Compile CLLE Validity Checking Program */
CRTBNDCL PGM(&CURLIB/CHKLIBS) SRCFILE(&CURLIB/QCLLESRC) SRCMBR(CHKLIBS) +
         DBGVIEW(*SOURCE)

/* 5. Create Custom Command with Validity Checking Program */
CRTCMD CMD(&CURLIB/EXTRPGSRC) PGM(&CURLIB/EXTRPGCL) +
       SRCFILE(&CURLIB/QCMDSRC) SRCMBR(EXTRPGSRC) +
       VLDPGM(&CURLIB/CHKLIBS) +
       TEXT('Extract RPG Source to Consolidated PF')
```

6. Comprehensive Test Plan

Test ID	Test Scenario	Command Invocation	Expected Behavior
TC-01	Single invalid library	`EXTRPGSRC LIB(NO_SUCH_LIB)`	`VLDPGM (CHKLIBS)` halts execution. Diagnostic: "Library NO_SUCH_LIB does not exist." `EXTRPGCL` is **not** called. Output PF unchanged.
TC-02	Mixed valid and invalid libraries	`EXTRPGSRC LIB(VALIDLIB BADLIB1 BADLIB2)`	`VLDPGM (CHKLIBS)` reports diagnostics for both `BADLIB1` and `BADLIB2`. Halts with `CPF0002`. Zero extraction performed.
TC-03	Unauthorized library	User lacks authority to library	`VLDPGM (CHKLIBS)` traps `CPF9802`. Reports "Not authorized to library ...". Halts execution.
TC-04	Single valid library	`EXTRPGSRC LIB(MYDEVLIB)`	`VLDPGM` passes. `EXTRPGCL` processes `MYDEVLIB`. All eligible RPG members extracted to `EXTRPGPF`.
TC-05	Multiple valid libraries	`EXTRPGSRC LIB(ERP HR SVC)`	All 3 libraries processed. Records consolidated into `EXTRPGPF` tagged with corresponding `SRCLIB`.
TC-06	Maximum (10) libraries	`EXTRPGSRC LIB(L1 L2 L3 L4 L5 L6 L7 L8 L9 L10)`	All 10 validated by `CHKLIBS` and extracted by `EXTRPGCL`. Summary displays 10 libraries processed.
TC-07	Exceed limit (>10 libraries)	Specifying 11 libraries	Command Analyzer rejects immediately with message `CPD0014` / `CPF0001` before `CHKLIBS` is called.
TC-08	Non-standard source file names	Member in `MYAPP/CUSTOMSRC` of type `RPGLE`	Discovered by `DSPFD` based on `MLFTYP='S'` and `MLSEU2='RPGLE'`. Extracted successfully.
TC-09	Diverse source types	Members with types `RPG`, `RPGLE`, `SQLRPGLE`, `CLLE`, `DDS`	`RPG`, `RPGLE`, `SQLRPGLE` extracted; `CLLE` and `DDS` safely ignored.
TC-10	Empty member	Source member with 0 lines	Handled smoothly by `INSERT ... SELECT`; 0 rows inserted for that member; process continues.
TC-11	Output Clear vs Append	`CLEAR(*NO)` vs `CLEAR(*YES)`	`CLEAR(*YES)` clears PF before run; `CLEAR(*NO)` appends records to existing data.

7. Data Verification Queries

After executing `EXTRPGSRC`, operators can audit the consolidated output PF using these SQL queries (via STRSQL, Run SQL Scripts in ACS, or QZDFMAPI):

```
sql
-- 1. Summary of extracted records by Library, File, and Source Type
SELECT SRCLIB, SRCFILE, SRCTYPE, COUNT(DISTINCT SRCMBR) AS TOTAL_MEMBERS, COUNT(*) AS
TOTAL_LINES
FROM QGPL.EXTRPGPF
GROUP BY SRCLIB, SRCFILE, SRCTYPE
ORDER BY SRCLIB, SRCFILE, SRCTYPE;

-- 2. Inspect extracted source lines for a specific program member
SELECT SRCSEQ, SRCDAT, SRCDTA
FROM QGPL.EXTRPGPF
WHERE SRCLIB = 'ERP' AND SRCMBR = 'CUSTINQ'
ORDER BY SRCSEQ;

-- 3. Verify no non-RPG source types slipped in
SELECT DISTINCT SRCTYPE
FROM QGPL.EXTRPGPF;

-- 4. Audit timestamp and total record count
SELECT COUNT(*) AS TOTAL_EXTRACTED_LINES, MIN(EXTRTIME) AS RUN_START, MAX(EXTRTIME) AS
RUN_END
FROM QGPL.EXTRPGPF;
```

8. User Confirmation

The plan now fully integrates:

- `CRTCMD ... VLDPGM(CHKLIBS)` pre-execution validation.
- Precise error signaling (`CPD0006` `*DIAG` + `CPF0002` `*ESCAPE`) that prevents `EXTRPGCL` from being called if any library is missing.
- Dynamic SQL alias extraction to guarantee zero level checks (`CPF4131`) across varying record lengths.
- Complete compilation and test plan.

Upon confirmation, all complete, compilable source members (`EXTRPGSRC.CMD`, `CHKLIBS.CLLE`, `EXTRPGCL.CLLE`, `EXTRPGRG.SQLRPGLE`, `EXTRPGPF.PF`, `EXTRPGPF.SQL`, and `BUILD.CLLE`) will be generated.